

*Python*基础





目录

01 Python安装

02 数据结构及对应“方法”

03 控制流语句的使用

04 字符串处理技巧

05 自定义函数详解

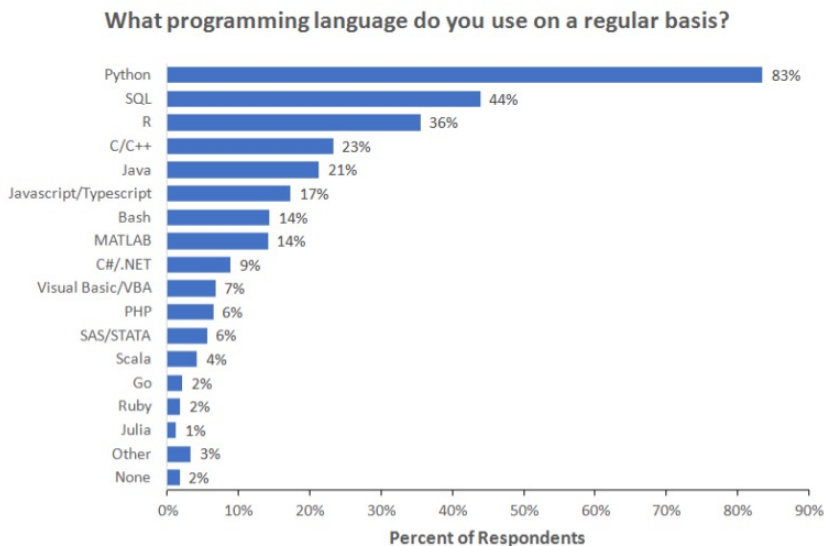
Python安装

Python的主要优点

- 设计严格：可读性强，易于维护
- 库很丰富：能够广泛应用于各种问题的场景
- 免费开源：具有高度可移植性，可在各类平台顺利工作
- 可扩展性：能够对R、C/C++等多种语言进行调用
- 可嵌入性：能够被集成到C/C++中，从而给程序用户提供脚本功能

Python安装

Python的主要优点



Note: Data are from the 2018 Kaggle Machine Learning and Data Science Survey. You can learn more about the study here: <http://www.kaggle.com/kaggle/kaggle-survey-2018>. A total of 18827 respondents answered the question.

Python安装

Anaconda简介

Anoconda属于专门用于科学计算的Python发行版，支持Windows、Linux和Mac系统，可以很方便地解决多版本Python并存、切换以及各种第三方模块安装的问题。

<https://www.anaconda.com/download/>

更重要的是，当你下载并安装好Anoconda后，它就已经集成了上百个科学计算的第三方模块，用户需要时，直接导入即可，不用再去下载。



ANACONDA

Python安装

Anaconda简介

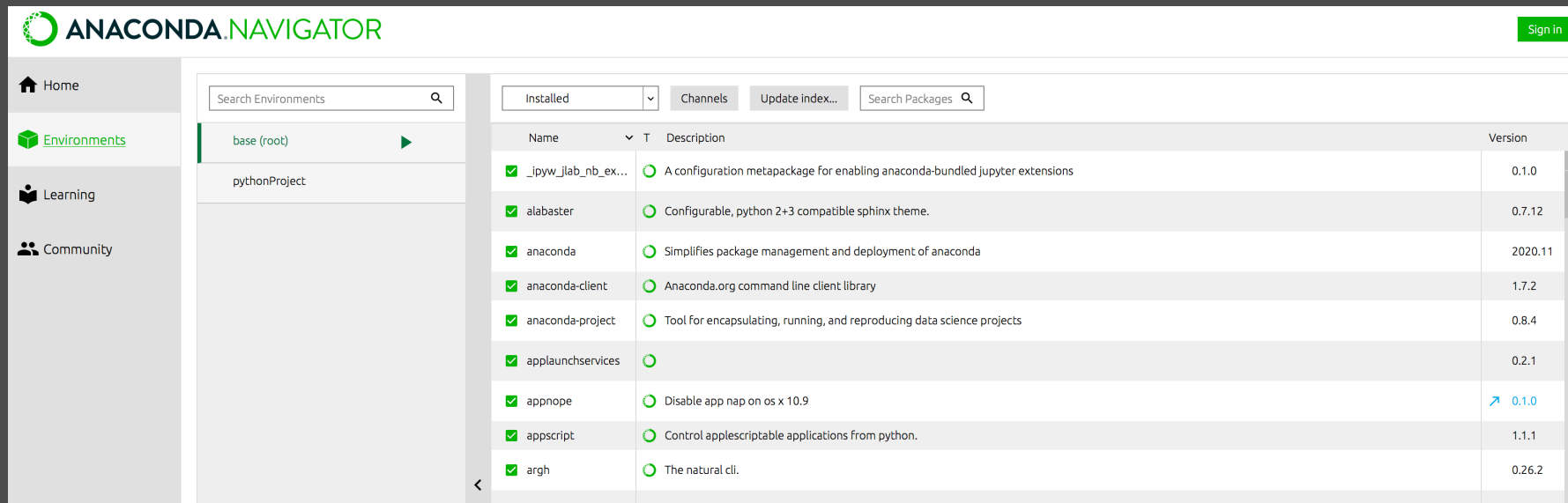
The screenshot displays the Anaconda Navigator desktop application. On the left is a sidebar with navigation links: Home, Environments, Learning, and Community. The main area is titled 'Applications on' with a dropdown menu set to 'base (root)' and a 'Channels' button. Below this, a grid of application tiles is shown, each with an icon, name, version, description, and a button to launch or install the application.

Application	Version	Description	Action
Datalore		Online Data Analysis Tool with smart coding assistance by JetBrains. Edit and run your Python notebooks in the cloud and share them with your team.	Launch
IBM Watson Studio Cloud		IBM Watson Studio Cloud provides you the tools to analyze and visualize data, to cleanse and shape data, to create and train machine learning models. Prepare data and build models, using open source data science tools or visual modeling.	Launch
JupyterLab	2.2.6	An extensible environment for interactive and reproducible computing, based on the Jupyter Notebook and Architecture.	Launch
Notebook	6.1.4	Web-based, interactive computing notebook environment. Edit and run human-readable docs while describing the data analysis.	Launch
PyCharm Professional	2020.3.1	A full-fledged IDE by JetBrains for both Scientific and Web Python development. Supports HTML, JS, and SQL.	Launch
Qt Console	4.7.7	PyQt GUI that supports inline figures, proper multiline editing with syntax highlighting, graphical calltips, and more.	Launch
Spyder	4.1.5	Scientific PYTHON Development Environment. Powerful Python IDE with advanced editing, interactive testing, debugging and introspection features	Launch
Glueviz	1.0.0	Multidimensional data visualization across files. Explore relationships within and among related datasets.	Install
Orange 3	3.26.0	Component based data mining framework. Data visualization and data analysis for novice and expert. Interactive workflows with a large toolbox.	Install
RStudio	1.1.456	A set of integrated tools designed to help you be more productive with R. Includes R essentials and notebooks.	Install

At the bottom left, there is a 'NUMFOCUS' logo with the tagline 'OPEN CODE • BETTER SCIENCE' and a 'Donate' button.

Python安装

Anaconda简介



The screenshot displays the Anaconda Navigator application interface. The top bar features the "ANACONDA.NAVIGATOR" logo and a "Sign in" button. The left sidebar contains navigation links for Home, Environments, Learning, and Community. The main panel is divided into two sections: "Environments" on the left and "Channels" on the right. The "Environments" section shows a search bar and a list of environments, including "base (root)" and "pythonProject". The "Channels" section shows a list of installed packages with columns for Name, Description, and Version.

Name	Description	Version
✓ _ipyw_jlab_nb_ex...	⦿ A configuration metapackage for enabling anaconda-bundled jupyter extensions	0.1.0
✓ alabaster	⦿ Configurable, python 2+3 compatible sphinx theme.	0.7.12
✓ anaconda	⦿ Simplifies package management and deployment of anaconda	2020.11
✓ anaconda-client	⦿ Anaconda.org command line client library	1.7.2
✓ anaconda-project	⦿ Tool for encapsulating, running, and reproducing data science projects	0.8.4
✓ applaunchservices	⦿	0.2.1
✓ appnope	⦿ Disable app nap on os x 10.9	0.1.0
✓ appscript	⦿ Control applescriptable applications from python.	1.1.1
✓ argh	⦿ The natural cli.	0.26.2

Python安装

Anaconda简介

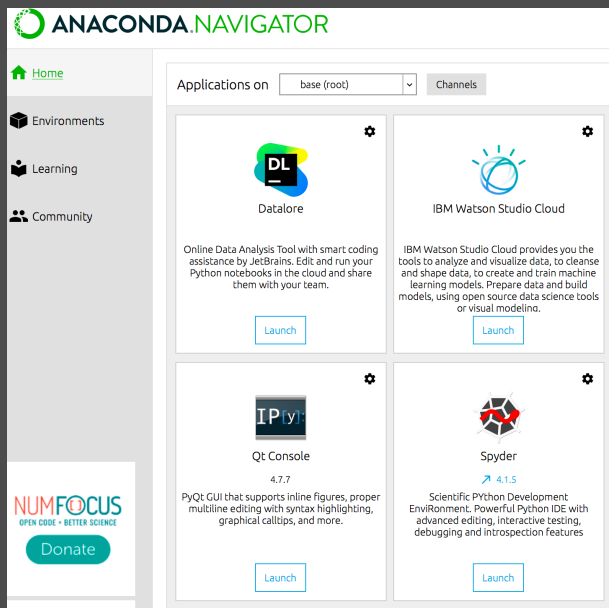
The screenshot displays the Anaconda Navigator web application. At the top, the header includes the 'ANACONDA.NAVIGATOR' logo and a 'Sign in' button. A left sidebar contains navigation links for 'Home', 'Environments', 'Learning', and 'Community'. Below the sidebar, there's a 'NUMFOCUS' logo with the tagline 'OPEN CODE • BETTER SCIENCE' and a 'Donate' button, followed by 'Support the OSS Community' and 'Documentation' links. The main content area features a filter bar with tabs for 'Documentation (26)', 'Training (1)', 'Video (21)', and 'Webinar (20)', along with a search bar. The central grid shows 24 resource cards, each with an icon, title, and a 'Read' button. The cards are organized as follows:

Row	Card 1	Card 2	Card 3	Card 4	Card 5	Card 6	Card 7
1	Python Tutorial	Python Reference	Anaconda Package List	Pandas Documentation	Numpy Documentation	Scipy Documentation	Matplotlib Documentation
2	Bokeh User Guide	Anaconda Cloud Documentation	Anaconda Documentation	Anaconda Navigator Documentation	The Comprehensive R Archive Network (CRAN)	The Python Package Index (PyPI)	Dask documentation
3	Conda & Conda-Build	Jupyter documentation	Spyder documentation	VSCode (python)	Orange documentation	Scikit-Learn Documentation	PyTorch Documentation

Python安装

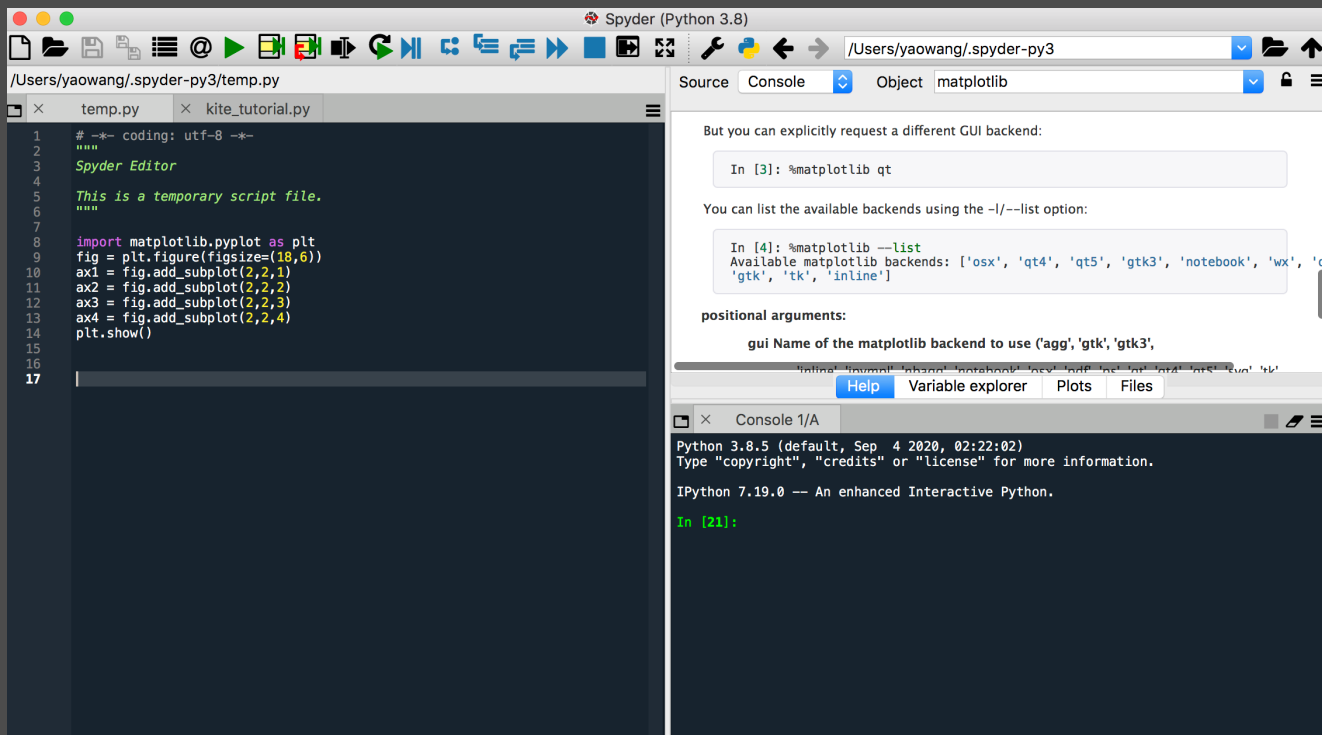
Spyder简介

Spyder是使用Python语言进行科学计算的IDE。它结合了综合开发工具的高级编辑、分析、调试功能以及多种可视化功能，作用类似于R语言中的Rstudio。Anaconda可直接启动Spyder。



Python安装

Spyder简介



数据结构及对应“方法”

如何将值输入给一个变量呢？

等号赋值法

```
A = 10  
B = A  
C = A + B  
D = max([A,B,33])  
print(A,B,C,D)
```

```
out:  
10 10 20 33
```

input函数输入法

```
name = input()  
age = input('请输入年龄：')  
print(name, age)
```

张三

请输入年龄：27

```
out:  
张三 27
```

数据结构及对应“方法”

两种输入方法的注意事项

- ✦ 两种方法中，等号左侧必须是一个具体的变量名
- ✦ 等号赋值法中，等号右侧可以是常量、变量、表达式或函数
- ✦ input函数的作用可以形成交互式效果，调用者必须输入具体的值
- ✦ input函数中，不管输入什么值，都将以字符串形式赋予变量

数据结构及对应“方法”

问题背景

如上介绍的两种赋值方法中，都是给变量传递单个值，如何同时给变量传递多个值呢？这就需要用到Python中容器的概念，即数据结构。

基本的数据结构包括：

列表

元组

字典

数据结构及对应“方法”

列表

- ✦ 列表的构造是通过英文状态下的方括号完成的，即[]
- ✦ 列表中的元素不受任何限制，即可以存放数值、字符串及其他数据结构的内容
- ✦ 列表是一种序列，即每个列表元素是按照顺序存入的
- ✦ 列表是一种可变类型的数据结构，即可以对列表的元素做修改

```
ll = [1,3.14,False,'HUAWEI','2019-05-03',[1,3,5,7]]  
list1 = ['张三','男',33,'江苏','硕士','已婚',['身高178','体重72']]  
list2 = ['江苏','安徽','浙江','上海','山东','山西','湖南','湖北']  
list3 = [1,10,100,1000,10000]
```

数据结构及对应 “方法”

列表的索引技术--正向单索引

正向单索引指的是只获取列表中的某一个元素，并且是从左到右的方向索取对应位置下的元素，可以使用[index]表示。

需要注意的是，索引值index是从0开始的，所以索引值与实际元素的位置正好差1。

```
list1 = ['张三','男',33,'江苏','硕士','已婚',['身高178','体重72']]  
# 取出第一个元素  
print(list1[0])  
# 取出第四个元素  
print(list1[3])  
# 取出最后一个元素  
print(list1[6])  
# 取出 “体重72”这个值  
print(list1[6][1])
```

out:

张三

江苏

['身高178', '体重72']

体重72

数据结构及对应 “方法”

列表的索引技术--负向单索引

负向单索引是指在正向单索引的基础上添加一个负号 “-” ，所表达的含义是从右向左的方向获取元素，可以用[-index]表示。

需要注意的是，负索引index是从-1开始的。

```
list1 = ['张三','男',33,'江苏','硕士','已婚',['身高178','体重72']]  
# 取出最后一个元素  
print(list1[-1])  
# 取出 “身高178”这个值  
print(list1[-1][0])  
# 取出倒数第三个元素  
print(list1[-3])
```

out:

['身高178','体重72']

身高178

硕士

数据结构及对应 “方法”

列表的索引技术--有限切片

切片索引指的是按照固定的步长，连续取出多个元素，可以用[start:end:step]表示。其中，start指定索引的起始位置；end指定索引的终止位置（注意，end位置的元素取不到！）；step指步长，默认为1，表示逐个取出一连串的元素。

```
list2 = ['江苏','安徽','浙江','上海','山东','山西','湖南','湖北']
```

```
# 取出 “浙江” 至 “山西” 四个元素
```

```
print(list2[2:6])
```

```
# 取出 “安徽” 、 “上海” 、 “山西” 三个元素
```

```
print(list2[1:6:2])
```

```
# 取出 “山西” 、 “湖南” 两个元素
```

```
print(list2[-3:-1])
```

out:

```
['浙江','上海','山东','山西']
```

```
['安徽','上海','山西']
```

```
['山西','湖南']
```

数据结构及对应 “方法”

列表的索引技术--无限切片

无限索引是指在切片过程中不限定起始元素的位置或终止元素的位置，甚至起始和终止元素的位置都不限定，可以用[::step]表示。第一个冒号是指从列表的第一个元素开始获取；第二个冒号是指到最后一个元素结束（包含最后一个元素值）。

```
list2 = ['江苏','安徽','浙江','上海','山东','山西','湖南','湖北']  
# 取出头三个元素  
print(list2[:3])  
# 取出最后三个元素  
print(list2[-3:])  
# 取出奇数位置的元素  
print(list2[::2])
```

out:

```
['江苏','安徽','浙江']  
['山西','湖南','湖北']  
['江苏','浙江','山东','湖南']
```

数据结构及对应 “方法”

列表元素的增加

如需给列表增加新的元素，可以使用其对应的三种方法，即append、extend和insert。

append：往列表尾部增加一个元素；

extend：可以往列表尾部增加多个元素；

insert：可以往列表的任意位置增加一个元素；

```
list3 = [1,10,100,1000,10000]
# 在列表末尾添加数字2
list3.append(2)
# 在列表末尾添加20,200,2000,20000四个值
list3.extend([20,200,2000,20000])
# 在数字10后面增加11这个数字
list3.insert(2,11)
# 在10000后面插入['a','b','c']
list3.insert(6,['a','b','c'])
```

数据结构及对应 “方法”

列表元素的增加

如需给列表增加新的元素，可以使用其对应的三种方法，即append、extend和insert。

append：往列表尾部增加一个元素；

extend：可以往列表尾部增加多个元素；

insert：可以往列表的任意位置增加一个元素；

```
list3 = [1,10,100,1000,10000]
# 在列表末尾添加数字2
list3.append(2)
# 在列表末尾添加20,200,2000,20000四个值
list3.extend([20,200,2000,20000])
# 在数字10后面增加11这个数字
list3.insert(2,11)
# 在10000后面插入['a','b','c']
list3.insert(6,['a','b','c'])
```

数据结构及对应“方法”

列表元素的删除

如需将列表元素做删除操作，可以使用其对应的三种方法，即pop、remove和clear。

pop：根据指定的索引值删除对应的元素，如果不指定索引值，默认删除最后一个元素；

remove：根据指定的值删除对应的元素；

clear：列表元素的清空；

```
# 继续基于列表ls3的结果做删除操作
# 删除list3中20000这个元素
list3.pop()
# 删除list3中11这个元素
list3.pop(2)
# 删除list3中的['a', 'b', 'c']
list3.remove(['a', 'b', 'c'])
# 删除list3中所有元素
list3.clear()
```

数据结构及对应“方法”

列表的元素的修改

如果列表元素值存在错误该如何修改呢？不像之前介绍的列表元素的增加和删除，修改操作没有对应的“方法”。但可以使用“取而代之”的思想实现元素的修改，语法如下：

```
mylist[index] = newValue
```

```
list4 = ['洗衣机','冰响','电视机','电脑','空调']  
# 将“冰响”修改为“冰箱”  
print(list4[1])  
list4[1] = '冰箱'  
print(list4)
```

out:

冰响

['洗衣机','冰箱','电视机','电脑','空调']

数据结构及对应 “方法”

列表的其他常用 “方法”

除了上面介绍的列表元素增加和删除所涉及的 “方法” 外，还有其他 “方法”，如排序、计数、查询位置、逆转等。

```
list5 = [7,3,9,11,4,6,10,3,7,4,4,3,6,3]
# 计算列表中元素3的个数
print(list5.count(3))
# 找出元素6所在的位置
print(list5.index(6))
# 列表元素的颠倒
list5.reverse()
print(list5)
# 列表元素的降序
list5.sort(reverse=True)
print(list5)
```

out:

4

5

[3, 6, 3, 4, 4, 7, 3, 10, 6, 4, 11, 9, 3, 7]

[11, 10, 9, 7, 7, 6, 6, 4, 4, 4, 3, 3, 3, 3]

数据结构及对应“方法”

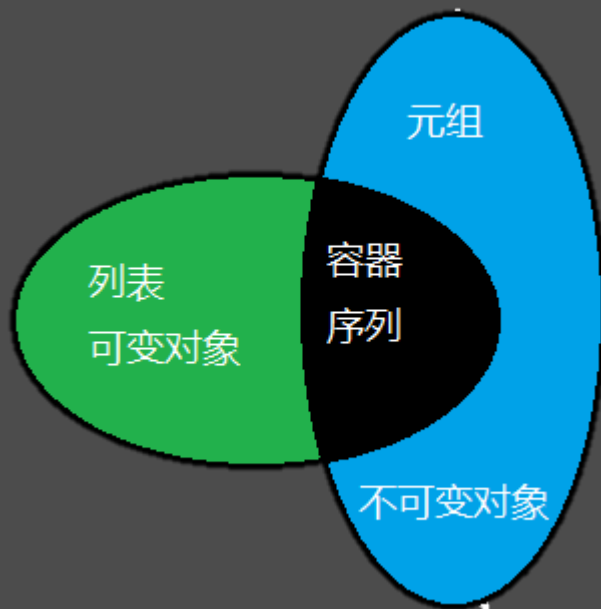
元组

- ✦ 元组通过英文状态下的圆括号构成，即()
- ✦ 元组中的元素不受任何限制，即可以存放数值、字符串及其他数据结构的内容
- ✦ 元组是一种序列，即每个元素都是按照顺序存入的，故列表中的索引技巧在元组中适用
- ✦ 元组是不可变类型的数据结构，即无法对元组的元素做修改

```
tt = (1,3.14,False,'HUAWEI','2019-05-03',[1,3,5,7],(10,20,100))  
t1 = ('张三','男',33,'江苏','硕士','已婚',['身高178','体重72'])  
t2 = ('江苏','安徽','浙江','上海','山东','山西','湖南','湖北')  
t3 = (1,10,100,1000,10000)
```


数据结构及对应“方法”

元组与列表的差异



数据结构及对应 “方法”

元组的可用 “方法”

由于元组只是存储数据的不可变容器，因此其只有两种可用的“方法”，分别是count和index。

```
t = ('a','d','z','a','d','c','a')  
# 计数  
print(t.count('a'))  
# 元素位置  
print(t.index('c'))
```

out:

3

5

数据结构及对应 “方法”

字典

- ✦ 构造字典对象需要使用大括号表示，即{}
- ✦ 字典元素都是以键值对的形式存在，并且键值对之间用英文状态下的冒号隔开，即key:value
- ✦ 键在字典中是唯一的，不能有重复
- ✦ 字典不再是序列（故无法通过索引值获取字典元素），但其与列表一样，都是可变类型的数据结构

```
dict1 = {'姓名': '张三', '年龄': 33, '性别': '男', '子女': {'儿子': '张四', '女儿': '张美'},  
        '兴趣': ['踢球', '游泳', '唱歌']}  
dict2 = {'电影': ['三傻大闹宝莱坞', '大话西游之大圣娶亲', '疯狂动物城'],  
        '导演': ['拉吉库马尔·希拉尼', '刘镇伟', '拜伦·霍华德'], '评分': [9.1, 9.2, 9.2]}
```

数据结构及对应“方法”

字典元素的获取

由于字典不再是序列，就无法借助于列表或元组中的索引方法返回字典中的元素。如需返回字典的元素，可以使用键索引或者get方法。

```
dict1 = {'姓名': '张三', '年龄': 33, '性别': '男', '子女': {'儿子': '张四', '女儿': '张美'},  
        '兴趣': ['踢球', '游泳', '唱歌']}
```

```
# 取出年龄
```

```
print(dict1['年龄'])
```

```
# 取出子女中的儿子姓名
```

```
print(dict1['子女']['儿子'])
```

```
# 取出兴趣中的游泳
```

```
print(dict1['兴趣'][1])
```

out:

33

张四

游泳

数据结构及对应 “方法”

字典元素的增加

如需给字典增加新的元素，可以使用setdefault方法、update方法和键索引方法。

setdefault：以默认值的形式，给字典增加元素；

update：以字典更新的名义，给字典增加元素；

键索引：使用索引技巧增加元素；

```
dict1 = {'姓名': '张三', '年龄': 33, '性别': '男', '子女': {'儿子': '张四', '女儿': '张美'},  
        '兴趣': ['踢球', '游泳', '唱歌']}  
# 往字典dict1中增加户籍信息  
dict1.setdefault('户籍', '合肥')  
# 增加学历信息  
dict1.update({'学历': '硕士'})  
# 增加身高信息  
dict1['身高'] = 178
```

数据结构及对应“方法”

字典元素的删除

如需将字典元素做删除操作，可以使用其对应的三种方法，即pop、popitem和clear。

pop：根据键名称删除对应的元素；

popitem：任意删除字典中的某个元素；

clear：字典元素的清空；

```
# 删除字典中的户籍信息
dict1.pop('户籍')
# 删除字典中女儿的姓名
dict1['子女'].pop('女儿')
# 删除字典中的任意一个元素
dict1.popitem()
# 清空字典元素
dict1.clear()
```

数据结构及对应“方法”

字典元素的修改

如果字典元素值存在错误该如何修改呢？一种方法类似于列表中的“取而代之”，另一种方法则是使用update方法。

```
# 将学历改为本科
dict1.update({'学历': '本科'})
# 将年龄改为35
dict1['年龄'] = 35
# 将兴趣中的唱歌改为跳舞
dict1['兴趣'][2] = '跳舞'
```

数据结构及对应 “方法”

字典的其他常用 “方法”

除了上面介绍的字典元素增加和删除所涉及的 “方法” 外，还有其他 “方法”，如返回字典的键、值以及键值对。

```
dict2 = {'电影':['三傻大闹宝莱坞','大话西游之大圣娶亲','疯狂动物城'],  
        '导演':['拉吉库马尔·希拉尼','刘镇伟','拜伦·霍华德'], '评分':[9.1,9.2,9.2]}  
# 取出字典中的所有键  
print(dict2.keys())  
# 取出字典中的所有值  
print(dict2.values())  
# 取出字典中的所有键值对  
print(dict2.items())
```


控制流语句的使用

if分支示意图



控制流语句的使用

if分支语法介绍

```
if condition:  
    expression
```

```
if condition:  
    expression  
else:  
    expression
```

```
if condition:  
    expression  
elif condition:  
    expression  
else:  
    expression
```

控制流语句的使用

if分支注意细节

- ✦ 关键词 (if、elif和else) 所在的行末必须加上英文状态的冒号
- ✦ 冒号所在行的下一行必须缩进 (目的是为了代码的好看与好用)
- ✦ else后面永远不要写条件
- ✦ elif的正确写法 (错误写法 : elseif else if)

控制流语句的使用

if分支示例介绍

二分支：返回一个数的绝对值

```
x = -3
if x >= 0:
    print(x)
else:
    print(-1*x)

out:
3
```

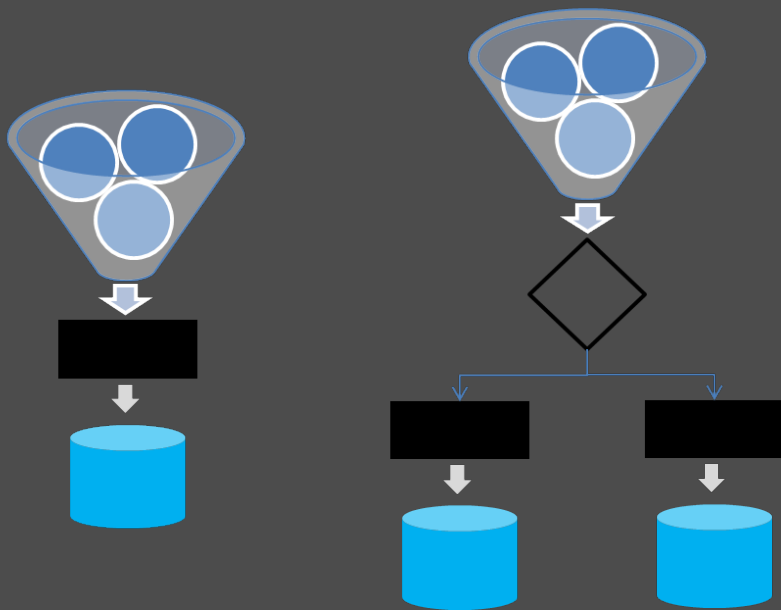
多分支：返回成绩对应的等级

```
score = 68
if score < 60:
    print('不及格')
elif score < 70:
    print('合格')
elif score < 80:
    print('良好')
else:
    print('优秀')

out:
合格
```

控制流语句的使用

for循环示意图



对于for循环来说，就是把可迭代对象中的元素（如列表中的每一个元素）通过漏斗的小口依次倒入之后的执行语句中。

控制流语句的使用

for循环语法介绍

```
for i in iterable:  
    if condition:  
        expression  
    elif condition:  
        expression  
    else:  
        expression
```

控制流语句的使用

for循环示例介绍

```
# 将列表中的每个元素做平方加1处理
list6 = [1,5,2,8,10,13,17,4,6]
result = []
for i in list6:
    y = i ** 2 + 1
    result.append(y)
print(result)
```

out:

```
[2, 26, 5, 65, 101, 170, 290, 17, 37]
```

控制流语句的使用

for循环示例介绍

```
# 计算1到100之间的偶数和
s1_100 = 0
for i in range(1,101):
    if i % 2 == 0:
        s1_100 = s1_100 + i
    else:
        pass
print('1到100之间的偶数和为%d'%s1_100)

out:
1到100之间的偶数和为2550
```

代码说明

- 进入循环之前须定义一个初始值为0的用于和的累加
- range(1,101)表示从1到101(上界取不到)的100个数字
- %和==用于余数运算和比较运算
- else子句使用关键词pass表示忽略，也可以省略掉else:和pass两行
- print输出部分使用了格式化的输出方法

控制流语句的使用

基于for循环的列表推导式

语法介绍

[expression for i in iterable if condition]

几点说明

- expression就是对每一个元素的具体操作表达式
- iterable是某个可迭代对象，如列表、元组或字符串等
- if condition是对每一个元素做分支判断，如果条件符合，则expression操作对应的元素

控制流语句的使用

列表推导式的示例介绍

```
# 对列表中的偶数做三次方减10的处理  
list7 = [3,1,18,13,22,17,23,14,19,28,16]  
result = [i ** 3 - 10 for i in list7 if i % 2 == 0]  
print(result)
```

out:

```
[5822, 10638, 2734, 21942, 4086]
```

控制流语句的使用

while循环与for循环的差异

- ✦ 绝大多数场景下，while循环与for循环是可以互相替代的
- ✦ 如果存在明确的被迭代对象（如列表、元组、字符串等），可优先使用for循环
- ✦ 如果不存在明确的被迭代对象，使用while循环将会更加地简单

控制流语句的使用

while循环语法介绍

```
while condition:  
    if condition1:  
        expression1  
    elif condition2:  
        expression2  
    else:  
        expression3
```

控制流语句的使用

while与for循环示例对比

使用for循环登录某手机银行APP – 已知登录总次数的情况
for i in range(1,6):

```
    user = input('请输入用户名 : ')
    password = int(input('请输入密码 : '))
    if (user == 'test') & (password == 123):
        print('登录成功 !')
        break
    else:
        if i < 5:
            print('错误 ! 您今日还剩%d次输入机会。' %(5-i))
        else:
            print('请24小时后再尝试登录 !')
```

out:

```
请输入用户名 : test
请输入密码 : 111
错误 ! 您今日还剩4次输入机会。
请输入用户名 : test
请输入密码 : 123
登录成功 !
```

使用while循环登录某邮箱账号 -- 无限次登录次数的情况
while True:

```
    user = input('请输入用户名 : ')
    password = int(input('请输入密码 : '))
    if (user == 'test') & (password == 123):
        print('登录成功 !')
        break
    else:
        print('您输入的用户名或密码错误 !')
```

out:

```
请输入用户名 : dag
请输入密码 : 111
您输入的用户名或密码错误 !
请输入用户名 : dasg
请输入密码 : 111
您输入的用户名或密码错误 !
请输入用户名 : test
请输入密码 : 123
登录成功 !
```

字符串处理技巧

字符串的构造方法

- ✦ 单引号：当字符串本身含有双引号时，可选择单引号构造字符串
- ✦ 双引号：当字符串本身含有单引号时，可选择双引号构造字符串
- ✦ 三引号：当字符串本身既含有单引号，又含有双引号时，可选择三引号构造字符串
- ✦ 三引号：当字符串比较长，需要多行显示时，也可以选择三引号构造字符串

字符串处理技巧

字符串的构造方法

单引号构造字符串

```
string1 = "commentTime":"2018-01-26 08:59:30","content":"包装良心！馅料新鲜！还会回购"
```

双引号构造字符串

```
string2 = "ymd:'2017-01-01',bWendu:'5°C',yWendu:'-3°C',tianqi:'霾~晴',fengxiang:'南风'"
```

三引号构造字符串

```
string3 = """nickName:"美美",'content':"环境不错，服务态度超好，就是有点小贵"""
```

```
string4 = """据了解，持续降雪造成安徽部分地区农房倒塌、种植养殖业大棚损毁，  
其中合肥、马鞍山、铜陵3市倒塌农房8间、紧急转移安置8人。"""
```

字符串处理技巧

字符串的常用方法

方法	使用说明	方法	使用说明
string[start:end:step]	字符串的切片	string.replace	字符串的替换
string.split	字符串的分割	sep.join	将可迭代对象按sep分割符拼接为字符串
string.strip	删除首尾空白	string.lstrip	删除字符串左边空白
string.rstrip	删除字符串右边空白	string.count	对字符串的子串计数
string.index	返回子串首次出现的位置	string.find	返回子串首次出现的位置（找不到返回-1）
string.startswith	字符串是否以什么开头	string.endswith	字符串是否以什么结尾

字符串处理技巧

字符串的常用方法的示例

```
# 获取身份证号码中的出生日期
print('123456198901017890'[6:14])
# 将手机号中的中间四位替换为四颗星
tel = '13612345678'
print(tel.replace(tel[3:7], '****'))
# 将邮箱按@符分隔开
print('12345@qq.com'.split('@'))
# 将Python的每个字母用减号连接
print('-'.join('Python'))
# 删除"今天星期日"的首尾空白
print("今天星期日".strip())
# 删除"今天星期日"的左边空白
print("今天星期日".lstrip())
# 删除"今天星期日"的右边空白
print("今天星期日".rstrip())
```

out:

```
19890101
136****5678
['12345', 'qq.com']
P-y-t-h-o-n
今天星期日
今天星期日
今天星期日
```

字符串处理技巧

字符串的常用方法的示例

```
# 计算子串“中国”在字符串中的个数
string5 = '中国方案引领世界前行，展现了中国应势而为、勇于担当的作用！'
print(string5.count('中国'))
# 查询"Python"单词所在的位置
string6 = '我是一名Python用户，Python给我的工作带来了许多便捷。'
print(string6.index('Python'))
print(string6.find('Python'))
# 字符串是否以“2018年”开头
string7 = '2017年匆匆走过，迎来崭新的2018年'
print(string7.startswith('2018年'))
# 字符串是否以“2018年”结尾
print(string7.endswith('2018年'))
```

out:

2

4

4

False

True

什么是正则表达式？

- 它是一种包括普通字符（如英文字母）和特殊字符（如 “\” “^” “\$” 等）的字符串匹配模式
- 英文名称为 Regular Expression，常常简写为 regex, regexp 或 RE
- 它可以“玩”出很多花样，用于检查、提取或替换文本中某些特定的子文本
- 多种计算机语言都支持利用正则表达式进行字符串操作

在线教程

RUNOOB.COM

搜索.....

[首页](#) [HTML](#) [CSS](#) [JAVASCRIPT](#) [JQUERY](#) [VUE](#) [BOOTSTRAP](#) [PYTHON3](#) [PYTHON2](#) [JAVA](#) [C](#) [C++](#) [C#](#) [GO](#) [SQL](#) [本地书签](#)

正则表达式

正则表达式 - 教程

正则表达式 - 简介

正则表达式 - 语法

正则表达式 - 修饰符

正则表达式 - 元字符

正则表达式 - 运算符优先级

正则表达式 - 匹配规则

正则表达式 - 示例

正则表达式 - 在线工具

正则表达式 - 简介 →

正则表达式 - 教程

正则表达式(Regular Expression)是一种文本模式，包括普通字符（例如，a 到 z 之间的字母）和特殊字符（称为"元字符"）。

正则表达式使用单个字符串来描述、匹配一系列匹配某个句法规则的字符串。

正则表达式是繁琐的，但它是强大的，学会之后的应用会让你除了提高效率外，会给你带来绝对的成就感。只要认真阅读本教程，加上应用的时候进行一定的参考，掌握正则表达式不是问题。

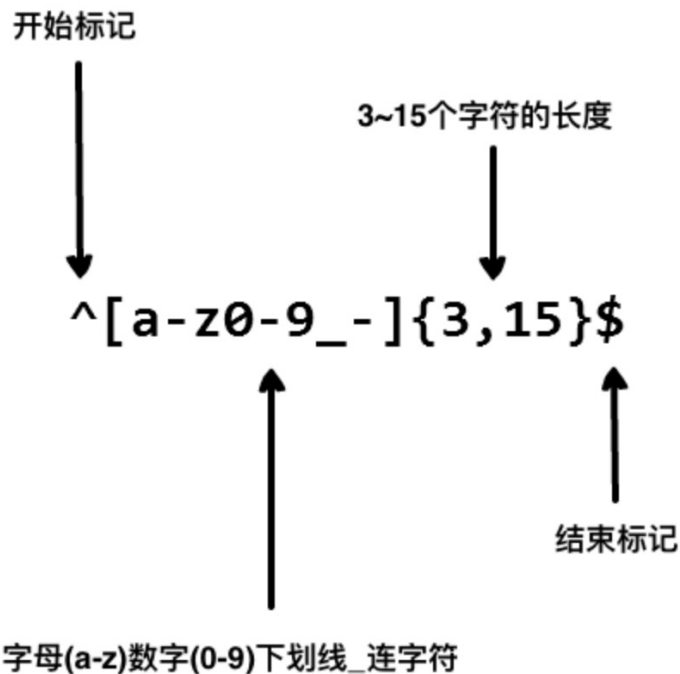
许多程序设计语言都支持利用正则表达式进行字符串操作。

[现在开始学习正则表达式！](#)

实例

<https://www.runoob.com/regexp/regexp-tutorial.html>

一个简单例子



以上的正则表达式可以匹配 **runoob**、**runoob1**、**run-oob**、**run_oob**，但不匹配 **ru**，因为它包含的字母太短了，小于 3 个无法匹配

在线测试

[菜鸟工具](#) [WEB 在线编辑器](#) [SVG 在线编辑器](#) [实例归档](#) [菜鸟教程](#) [Q](#)

正则表达式在线测试

[生成代码](#)

[测试匹配](#)

/

`[A-Za-z0-9\._]+@[A-Za-z0-9]+\.(com|org|edu)`

/g

修饰符 ▾

语法参考 ▾

yao.s.wang@gmail.com

共找到 1 处匹配:
yao.s.wang@gmail.com

字符串处理技巧

三个重要的函数之匹配查询函数

`findall(pattern, string, flags=0)`

pattern : 指定需要匹配的正则表达式。

string : 指定待处理的字符串。

flags : 指定匹配模式，常用的值可以是re.I、re.M、re.S和re.X。re.I的模式是让正则表达式对大小写不敏感；re.M的模式是让正则表达式可以多行匹配；re.S的模式指明正则符号可以匹配任意字符，包括换行符\n；re.X模式允许正则表达式可以写得更加详细，如多行表示、忽略空白字符、加入注释等。

字符串处理技巧

三个重要的函数之匹配替换函数

`sub(pattern, repl, string, count=0, flags=0)`

pattern : 同findall函数中的pattern。

repl : 指定替换成的新值。

string : 同findall函数中的string。

count : 用于指定最多替换的次数，默认为全部替换。

flags : 同findall函数中的flags。

字符串处理技巧

三个重要的函数之匹配分割函数

```
split(pattern, string, maxsplit=0, flags=0)
```

pattern : 同findall函数中的pattern。

maxsplit : 用于指定最大分割次数，默认为全部分割。

string : 同findall函数中的string。

flags : 同findall函数中的flags。

字符串处理技巧

常用的正则符号

符号	含义	示例
.	可以匹配任意字符，但不包含换行符 '\n'	Pyt.on → Pytmon
\	转义符，一般用于保留字符串中的特殊元字符	10\.3 → 10.3
	逻辑或	人a A → 人a或者人A
[]	用于匹配的一组字符	m[aA]n → man或者mAn
\d与\D	\d匹配任意数字，\D代表所有非\d	今天\d号 → 今天3号
\s与\S	\s匹配任意空白字符，\S代表所有非\s	你\s好 → 你 好
\w与\W	\w匹配字母数字和下划线，\W代表所有非\w	P\wy → Pay或者P3y
*	匹配前一个字符0到无穷次	OK* → O或者OK或者OKK
+	匹配前一个字符1到无穷次	OK+ → OK或者OKK
?	匹配前一个字符0到1次	OK? → O或者OK
{m}	匹配前一个字符m次	OK{3} → OKKK
{m,n}	匹配前一个字符m到n次	OK{1,2} → OK或者OKK
(.*)	用于分组，默认返回括号内的匹配内容	可见后续案例

字符串处理技巧

正则表达式应用的示例

```
# 导入用于正则表达式的re模块
import re
```

```
# 取出字符串中所有的天气状态
```

```
string8 = "{ymd:'2018-01-01',tianqi:'晴',aqiInfo:'轻度污染'}, {ymd:'2018-01-02',tianqi:'阴~小雨',aqiInfo:'优'}, {ymd:'2018-01-03',tianqi:'小雨~中雨',aqiInfo:'优'}, {ymd:'2018-01-04',tianqi:'中雨~小雨',aqiInfo:'优'} "
```

```
print(re.findall("tianqi:'(.*?)'", string8))
```

out:

```
['晴', '阴~小雨', '小雨~中雨', '中雨~小雨']
```

字符串处理技巧

正则表达式应用的示例

取出所有含O字母的单词

```
string9 = 'Together, we discovered that a free market only thrives when there are rules to ensure competition and fair play, Our celebration of initiative and enterprise '
```

```
print(re.findall('\w*o\w*',string9, flags = re.I))
```

将标点符号、数字和字母删除

```
string10 = '据悉，这次发运的4台蒸汽冷凝罐属于国际热核聚变实验堆（ITER）项目的核二级压力设备，先后完成了压力试验、真空试验、氦气检漏试验、千斤顶试验、吊耳载荷试验、叠装试验等验收试验。'
```

```
print(re.sub('[ , . 、 a-zA-Z0-9 ( ) ]', '', string10))
```

字符串处理技巧

正则表达式应用的示例

```
# 将每一部分的内容分割开
string11 = '2室2厅 | 101.62平 | 低区/7层 | 朝南 \n 上海未来 - 浦东 - 金杨 - 2005年建'
split = re.split('[-\\|\\n]', string11)
print(split)

split_strip = [i.strip() for i in split]
print(split_strip)
```

out:

```
['2室2厅', '101.62平', '低区/7层', '朝南', '上海未来', '浦东', '金杨', '2005年建']
['2室2厅', '101.62平', '低区/7层', '朝南', '上海未来', '浦东', '金杨', '2005年建']
```

自定义函数详解

匿名函数的定义

匿名函数，可以用lambda关键字定义。通过lambda构造的函数可以没有名称，最大特点是“一气呵成”，即在自定义匿名函数时，所有代码只能在一行内完成。



自定义函数详解

匿名函数的语法

```
lambda parameters : function_expression
```

lambda为匿名函数的关键起始词

parameters是函数的形参，多个参数之间用英文状态的逗号隔开

function_expression为具体的函数体

自定义函数详解

匿名函数的示例

```
# 统计列表中每个元素的频次
list6 = ['A','A','B','A','A','B','C','B','C','B','B','D','C']

# 构建空字典，用于频次统计数据的存储
dict3 = {}
# 循环计算
for i in set(list6):
    dict3[i] = list6.count(i)
print(dict3)
```

out:

```
{'D': 1, 'B': 5, 'A': 4, 'C': 3}
```


自定义函数详解

匿名函数的示例

```
# 取出字典中的键值对
key_value = list(dict3.items())
print(key_value)
```

```
# 列表排序
key_value.sort()
print(key_value)
```

```
# 按频次高低排序
key_value.sort(key = lambda x : x[1], reverse=True)
print(key_value)
```

out:

```
[('D', 1), ('B', 5), ('A', 4), ('C', 3)]
[('A', 4), ('B', 5), ('C', 3), ('D', 1)]
[('B', 5), ('A', 4), ('C', 3), ('D', 1)]
```

自定义函数详解

自定义函数

虽然匿名函数很灵活，会在很多代码中遇到，但它的最大特点也是它的短板，即无法通过lambda函数构造一个多行而复杂的函数。为了弥补其缺陷，Python提供了另一个关键字def，可以构造逻辑复杂的自定义函数。



自定义函数详解

自定义函数的语法

```
def function_name(parameters):  
    function_expression  
    return(result)
```

def是define单词的缩写，为自定义函数的关键词

function_name为自定义的函数名称

parameters为自定义函数的形参，需要放在圆括号内

function_expression为具体的函数体

return用于返回函数的计算结果

自定义函数详解

自定义函数的示例

```
# 猜数字
def game(min,max):
    import random
    number = random.randint(min,max) # 随机生成一个需要猜的数字
    while True:
        guess = float(input('请在%d到%d之间猜一个数字: ' %(min, max)))

        # if分支判断下一轮应在什么范围内猜数字
        if guess < number:
            min = guess
            print('不好意思，你猜的数偏小了！请在%d到%d之间猜一个数！' %(min,max))
        elif guess > number:
            max = guess
            print('不好意思，你猜的数偏大了！请在%d到%d之间猜一个数！' %(min,max))
        else:
            print('恭喜你猜对了！')
            print('游戏结束！')
            break # 如果猜对，通过break关键词退出整个循环体
```

game(1,10)

请在1到10之间猜一个数字: 2

不好意思，你猜的数偏小了！请在2到10之间猜一个数！

请在2到10之间猜一个数字: 2.5

不好意思，你猜的数偏小了！请在2到10之间猜一个数！

请在2到10之间猜一个数字: 5

不好意思，你猜的数偏小了！请在5到10之间猜一个数！

请在5到10之间猜一个数字: 8

不好意思，你猜的数偏大了！请在5到8之间猜一个数！

请在5到8之间猜一个数字: 6

恭喜你猜对了！

游戏结束！

自定义函数详解

自定义函数的几种参数

- ✦ 必选参数：自定义函数时设定了无初始值的形参，在函数调用时，必须赋予对应的实参
- ✦ 默认参数：自定义函数时设定了有初始值的形参，在函数调用时，可以不用传值
- ✦ 可变参数：在不确定需设定多少个参数时，可以在参数前加一个星号，表示可变参数
- ✦ 关键字参数：与可变参数类似，所不同的是在参数前加两个星号，且参数需以key=value的形式传递

自定义函数详解

默认参数

```
# 计算1到n的p次方和
```

```
def square_sum(n, p = 2):  
    result = sum([i ** p for i in range(1,n+1)])  
    return(n,p,result)  
  
print('1到%d的%d次方和为%d ! ' %square_sum(200))  
print('1到%d的%d次方和为%d ! ' %square_sum(200,3))
```

out:

```
1到200的2次方和为2686700 !  
1到200的3次方和为404010000 !
```

自定义函数详解

可变参数

两个数的求和

```
def add(a,b):  
    s = sum([a,b])  
    return(a,b,s)
```

```
print('%d加%d的和为%d ! ' %add(10,13))
```

out:
10加13的和为23 !

任意个数的数据求和

```
def adds(*args):  
    print(args)  
    s = sum(args)  
    return(s)
```

```
print('和为%d!' %adds(10,13,7,8,2))  
print('和为%d!' %adds(7,10,23,44,65,12,17))
```

out:
(10, 13, 7, 8, 2)
和为40!
(7, 10, 23, 44, 65, 12, 17)
和为178!

自定义函数详解

关键字参数

电商平台用户信息注册

```
def info_collection(tel, birthday, **kwargs):  
    user_info = {} # 构造空字典, 用于存储用户信息  
    user_info['tel'] = tel  
    user_info['birthday'] = birthday  
    user_info.update(kwargs)  
    # 用户信息返回  
    return(user_info)
```

调用函数

```
info_collection(13612345678,'1990-01-01',nickname='月亮',gender  
= '女',edu = '硕士',income = 15000,add = '上海市浦东新区',interest =  
['游泳','唱歌','看电影'])
```

out:

```
{'nickname': '月亮', 'gender': '女',  
'edu': '硕士', 'income': 15000, 'add': '  
上海市浦东新区', 'interest': ['游泳', '  
唱歌', '看电影']}
```

```
{'add': '上海市浦东新区',  
'birthday': '1990-01-01',  
'edu': '硕士',  
'gender': '女',  
'income': 15000,  
'interest': ['游泳', '唱歌', '看电影'],  
'nickname': '月亮',  
'tel': 13612345678}
```